

AI Conversation System Feedback Documentation

Overview

This document explains how the feedback system works in the AI-Conversation Training platform created at the JRCZ Data Science Lab. The system aims to help students improve their communication skills for the workforce by talking to a virtual character (avatar), getting feedback afterward, and learning what went well and what could be better. They could also show this feedback to their teachers for better advice.

Some documentation that explains how other parts of the system work includes: -

The backend system architecture is described at: <https://jrcz-data-science-lab.github.io/ai-client-simulation-backend-documentation/>

The front-end avatar rendering explained at: <https://rhinestone-bovid-b00.notion.site/Rendering-an-animated-avatar-on-the-web-with-React-Three-Fiber-18576d3fef0d80879f15f5caab769a86>

How It Works: Students' Perspective:

When a student finishes talking to the avatar (for example, the client “Nina”), they click the “End Call” button. This tells the system to start generating feedback. After the button is clicked, the system sends a message to the backend asking for an evaluation of the session. This is done using the current username. While the system waits for feedback, a loading spinner appears to show that the process is running.

Once the feedback is ready, a pop-up (called a modal) appears. This modal includes: - **Assessment** – Shows if the student asked enough questions for each topic. - **Asked Questions** – Lists what the student asked. - **Suggested Questions** – Lists questions the student could have asked but didn’t.

The feedback is displayed in a clean table to make it easy to understand. Students can download their feedback as a PDF by clicking a button. The system uses jsPDF and jspdf-autotable to create the file. After reviewing or downloading the feedback, students are taken back to the home screen.

How the Feedback was Implemented

Step-by-Step Implementation

1. **Defining Feedback Metrics (metrics.py):** The project uses **Gordon's 11 Functional Health Patterns** as the basis for feedback metrics. These patterns are essentially categories of nursing assessment (e.g., Nutrition, Sleep, Coping), each with a set of key questions a student should ask a patient. To represent these, the backend includes a file called **metrics.py** (this can be found under the API folder), which stores the predefined questions under each health pattern category.
2. **Incorporating Metrics into the Backend:** To use these metrics in the feedback process, the backend provides an **API endpoint** to retrieve the health patterns and their questions. (You can also find this under the API folder in the main.py file.)

```
#end point for metric.py
@app.get("/metrics")
def fetch_metrics():
    """Endpoint to retrieve health assessment metrics."""
    return {"patterns": get_gordons_patterns()}
```

There is a code to evaluate responses and feedback in the same file.

```
@app.post("/evaluate")
async def evaluate_feedback(request: FeedbackRequest):
    feedback = evaluate_responses(request.username)

    if "error" in feedback:
        raise HTTPException(status_code=404, detail=feedback["error"])
    return feedback

def get_student_responses(username):
    """Retrieve student responses from the conversation memory file"""
    filename = f"conversationMemory/{username}_memory.txt"

    if not os.path.exists(filename):
        return []

    student_responses = []
    with open(filename, "r") as file:
        lines = file.readlines()
        for line in lines:
            if line.startswith("User: "): # Extract only student responses
                student_responses.append(line.replace("User: ", "").strip())

    return student_responses

def evaluate_responses(username):
    """Uses LLM to evaluate student responses based on health patterns."""
    student_responses = get_student_responses(username) or [] # Retrieve saved re
    if not student_responses:
        return {"error": "No recorded responses for this user."}
    patterns = get_gordons_patterns()
    feedback = {}

    for category in patterns:
        category_name = category['pattern']
```

3. In the frontend canvasUI.tsx, you will find the function that handles generating the feedback and downloading the feedback.

```
const handleDownloadFeedback = () => {
  if (!feedback) return;

  const doc = new jsPDF();
  doc.text("Health Assessment Feedback", 14, 10);
  autoTable(doc, {
    head: [["Category", "Assessment", "Missed Aspects", "Suggested Questions"]],
    body: Object.entries(feedback).map(([category, details]) => [
      category,
      details.Assessment,
      details["Missed Aspects"].join(", ") || "None",
      details["Suggested Questions"].join(", ") || "None",
    ]),
    startY: 20,
    theme: "grid",
    headStyles: { fillColor: [41, 128, 185] },
  });
  doc.save("feedback.pdf");
  setTimeout(() => navigate("/"), 100); // Ensures download starts before redirect
};
```

Figure 20 Function that manages feedback download.

```
const CanvasUI = ({ hidden }: CanvasUIProps) => {
  const handleEndCall = async () => {
    if (client.name !== "Nina") {
      alert("Feedback is only available for the client 'Nina'.");
      navigate("/"); // Redirect to the homepage
      return;
    }

    setLoadingFeedback(true);
    try {
      const res = await fetch("https://145.19.54.110/api/evaluate", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ username }), // Ensure that `username` is valid
      });

      if (!res.ok) {
        throw new Error(`Failed to fetch feedback. Status: ${res.status}`);
      }

      const data = await res.json(); // Assuming this is the response body, parsing as JSON
      console.log("Feedback data:", data); // For debugging purposes

      // Ensure the data is in the expected format
      if (data && typeof data === "object") {
        setFeedback(data); // Set the parsed feedback
        setShowFeedbackModal(true); // Show the modal after data is fetched
      }
    } catch (error) {
      console.error(error);
    }
  };
};
```

The next step is to rewrite the backend in a way that the goal of the application is retained and improvements can be added.